

Combining Learning-Based Locomotion Policy With Model-Based Manipulation for Legged Mobile Manipulators

Yuntao Ma^{1b}, Graduate Student Member, IEEE, Farbod Farshidian^{1b}, Takahiro Miki^{1b},
Joonho Lee^{1b}, and Marco Hutter^{1b}, Member, IEEE

Abstract—Deep reinforcement learning produces robust locomotion policies for legged robots over challenging terrains. To date, few studies have leveraged model-based methods to combine these locomotion skills with the precise control of manipulators. Here, we incorporate external dynamics plans into learning-based locomotion policies for mobile manipulation. We train the base policy by applying a random wrench sequence on the robot base in simulation and add the noisified wrench sequence prediction to the policy observations. The policy then learns to counteract the partially-known future disturbance. The random wrench sequences are replaced with the wrench prediction generated with the dynamics plans from model predictive control to enable deployment. We show zero-shot adaptation for manipulators unseen during training. On the hardware, we demonstrate stable locomotion of legged robots with the prediction of the external wrench.

Index Terms—Legged robots, mobile manipulation, reinforcement learning.

I. INTRODUCTION

LEGGED robots have potential to traverse rough unstructured terrain that are unreachable by traditional wheeled or tracked robots, utilize human-oriented infrastructures such as stairs and doors, and conduct various tasks by carrying task-specific payloads in harsh environments instead of human.

Mounting an arm and allowing the robot to manipulate in complex environments can largely extend their potentials. However, controlling legged robots with arms is inherently a high-dimensional, contact-rich, and often non-smooth control problem, and it is still an open problem. To realize these potentials, many different control methods have been proposed. There

are mainly two complementary directions: model-based control approaches and learning-based or data-driven approaches.

Model-based control algorithms such as Model Predictive Control (MPC) have demonstrated precise control of wheeled mobile manipulators [1] and robustness to external disturbances [2]. However, such methods face challenges for hybrid systems. In particular, for the legged mobile manipulators, the possible contact configurations drastically expand the search space of the trajectories, making the computational cost for optimal planning significantly higher and hard to meet real-time requirements. To this end, many prior works use template models to reduce the computational cost, such as using zero-moment-point criteria [3] or centroidal models [4]. With reduced systems, online planning of the motion trajectory becomes feasible on flat ground [5], [6]. Unfortunately, however, applying such methods for robots operating over rough terrain is not straightforward. There can be various disturbances, such as unplanned contacts and slipping, which make even a simple task like online end-effector tracking while walking difficult.

Model-free Reinforcement Learning (RL) has recently shown promising results to learn legged locomotion [7], [8], [9], and [10]. In contrast to the model-based methods, RL policies do not suffer from higher planning cost due to contacts, and learn to tackle unexpected contacts and slippage from environment interactions. Besides, a policy can be robustified by applying controlled disturbances during training. Existing works apply random external force and torques in the form of impulses [11], constant external wrench within the same episode [8], or adversarial disturbances [12] to make the policy robust against unobservable external disturbances and model mismatches.

Despite the simplicity and versatility of RL-based approaches, transferring a policy to diverse robot configurations [13] and distinct tasks [14] still requires additional works. Usually, it requires re-training, which is time-consuming, and the policy can typically only be deployed on the robot that the policy is trained with, for the same task. Some recent works explored combining RL with model-based control and showed improved transferability. Prior works trained an RL agent to output only the desired twist for the base [15], or the desired foothold on unstructured terrain [16]. The model-based leg controllers then track these outputs.

Similarly, in this work, we split the legged mobile manipulator controller into two parts: the base and the arm controllers, which are separately constructed. For the base controller, we extend [8] and [10] by training a policy for the legged system

Manuscript received September 9, 2021; accepted January 7, 2022. Date of publication January 18, 2022; date of current version January 27, 2022. This letter was recommended for publication by Associate Editor R. Lioutikov and Editor J. Kober upon evaluation of the reviewers' comments. This work was supported in part by the Max Planck ETH Center for Learning Systems, in part by the Swiss National Science Foundation (SNSF) through Projects 166232 and 188596, in part by the National Centre of Competence in Research Robotics (NCCR Robotics), and in part by the European Union's Horizon 2020 under Grant Agreement Number 852044. Moreover, this work has been conducted as part of ANYmal Research, a community to advance legged robotics. (Corresponding author: Yuntao Ma.)

The authors are with the Robotic Systems Lab, ETH Zürich, 8092 Zürich, Switzerland (e-mail: mayuntao94@gmail.com; farshidian@mavt.ethz.ch; tamiki@ethz.ch; jolee@ethz.ch; mahutter@ethz.ch).

This letter has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2022.3143567>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2022.3143567

that utilizes the external wrench (force and torque) observation to counteract disturbances. We further demonstrate that using wrench sequence prediction as observation improves the base RL policy's performance over using only the current wrench. A proposed wrench sequence generator facilitates training of the policy without the arm controller. For deployment, we implement the arm controller with MPC. It allows us to extract the wrench predictions for the base policy while controlling the arm precisely. The combined system demonstrates the capability of manipulation on rough terrain in hardware experiments. Additionally, since the training of the base policy is decoupled from the manipulator configurations, it can adapt to multiple arm setups without re-training. This zero-shot adaptation would not be feasible with a single controller of the entire system trained end-to-end with RL.

Our main contributions are:

- A pipeline that combines model-based manipulator control and base locomotion policies trained with RL for legged mobile manipulation.
- Experiments showing the base locomotion policies having improved performance by using wrench predictions from MPC.
- Zero-shot adaptation to multiple manipulator configurations on a legged robot platform for end-effector tracking.
- Validation of the combined control system on a robotic hardware. The legged mobile manipulator walks on rough terrain while performing manipulation tasks.

II. BACKGROUND

A. Model Predictive Control

MPC solves the following Optimal Control (OC) problem

$$\underset{\mathbf{u}(\cdot)}{\text{minimize}} \quad \phi(\mathbf{x}(t_f)) + \int_{t_s}^{t_f} l(\mathbf{x}(t), \mathbf{u}(t), t) dt, \quad (1)$$

$$\text{subject to} \quad \mathbf{x}(t_s) = \mathbf{x}_s, \quad (2a)$$

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t), \quad (2b)$$

$$\mathbf{g}(\mathbf{x}, \mathbf{u}, t) = \mathbf{0}, \quad (2c)$$

$$\mathbf{h}(\mathbf{x}, \mathbf{u}, t) \geq \mathbf{0}, \quad (2d)$$

where $\mathbf{x}(t)$, $\mathbf{u}(t)$ are the states and control inputs at time t respectively. MPC finds the minimum-cost (1) trajectory under the system dynamics $\mathbf{f}$, the equality constraints $\mathbf{g}$ and the inequality constraints $\mathbf{h}$. The output of MPC is the nominal trajectory $\mathbf{x}(t)$ and $\mathbf{u}(t)$. If the dynamics model is perfect and the terminal cost function $\phi(\mathbf{x}(t_f))$ captures the true terminal cost at t_f , the optimized state trajectories would be identical to the experienced state trajectories under the optimal input trajectories $\mathbf{u}(t)$ within the MPC horizon. In practice, $\mathbf{u}(t_s)$ is then applied to control the robot in the receding horizon fashion to allow re-planning.

If the generalized positions, velocities and accelerations of the articulated system are available in the optimized state trajectory, an inverse dynamics approach such as the recursive Newton-Euler algorithm (RNEA) can provide the generalized forces consistent with these motions.

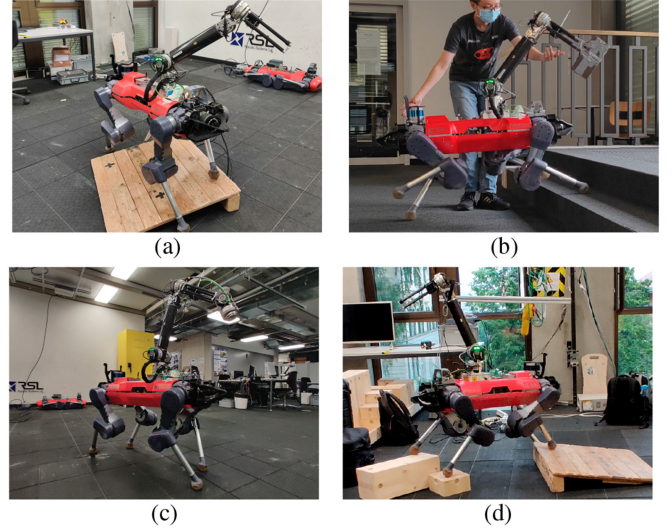


Fig. 1. Experiments with ALMA. (1a-1c) ALMA trotting with different manipulator configurations: no end-effector, fixed brick, a wrist weighing 1.8 kg. (1d) ALMA trotting on challenging terrain.

B. Reinforcement Learning for Robots

To apply RL for robotic control, a control problem is typically modeled as a discrete-time Markov Decision Process (MDP). At each timestep, the RL agent receives an observation $o_t \in \mathcal{O}$ from the environment and outputs an action $a_t \in \mathcal{A}$. The environment then updates with the action via the transition function $p(s_{t+1}|s_t, a_t)$, evaluates a reward $r_t \in \mathcal{R} : \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ and generates a new observation for the agent. The agent may act according to a stochastic policy $\pi_\theta(a_t|o_t, o_{t-1}, \dots)$. RL algorithms train the agent by updating the parameters θ of the policies to maximize the cumulative discounted rewards $\mathbb{E}[\sum_{t=k}^{\infty} \gamma^t r_t]$ through interactions with the environments, where k is the current timestep and γ is the discount factor. In practice, the robot's policy observation usually consists of its state measurements, sensor readings such as a height scan, internal states to enforce desired behavior patterns, and possibly a history of the above.

III. METHOD

A. Overview

Our method combines the precision and the planning capability of an MPC-based manipulation control and the robustness and the agility of an RL-based locomotion policy. Fig. 2 shows the schematics of our method. We decouple the manipulator control from base policy training by modeling the disturbances from the manipulator as predictable external wrenches. The base control policy takes predicted wrench sequences as input and uses them to counteract the external wrench resulting from the arm. During the training, we simulate external wrench predictions using a wrench sequence generator without having the manipulator or solving MPC. This drastically reduces the computational cost during the training process.

The floating-base arm is separately controlled by a whole-body MPC. When deploying on the robot, MPC takes the user commands and produces nominal state and input trajectories of the floating-base manipulator, allowing for wrench

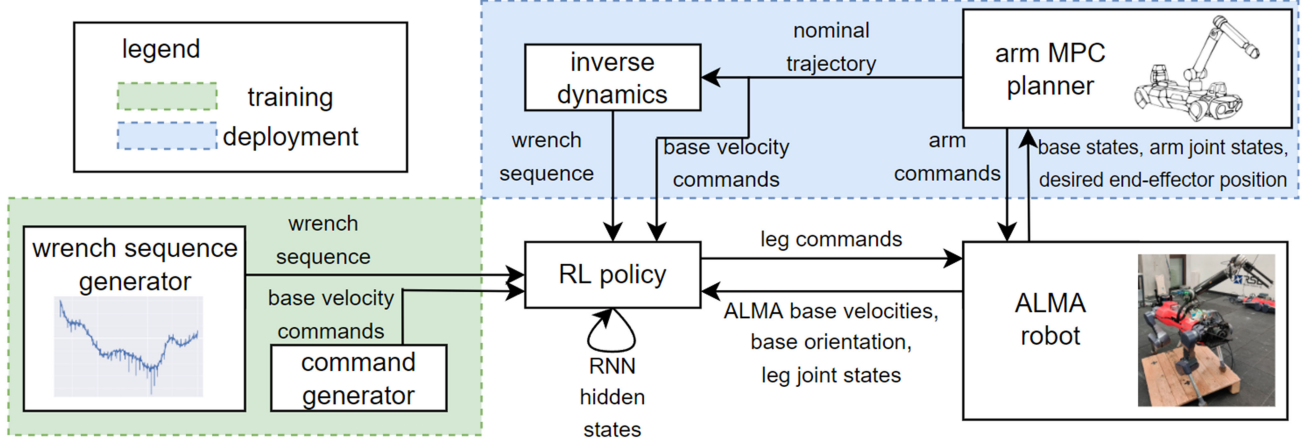


Fig. 2. Overview of the training and deployment pipeline. We train the RL policy with a wrench sequence generator and replace it with the wrench plan from MPC for deployment.

sequence predictions. In this work, we deployed our controller on ALMA platform [5], which consists of the Dynaarm manipulator mounted on ANYmal base [17] and we use Raisim physics engine [18] to simulate the robot during locomotion training.

The following sections describe the control problem for the manipulator system, the wrench sequence generator, and the structure of the RL agent.

B. MPC for the Manipulation System

Fig. 1(a)–1(c) illustrate our different manipulator setups. For the N -dof arm system, we model the system as a manipulator on a floating base.

$$\mathbf{x} = [p_x, p_y, p_z, r_x, r_y, r_z, \theta_1, \dots, \theta_N, \omega_1, \dots, \omega_N]^T \quad (3)$$

$$\mathbf{u} = [v_x, v_y, \omega_z, \alpha_1, \dots, \alpha_N]^T \quad (4)$$

The state, $\mathbf{x}$, consists of the base pose, manipulator joint positions θ , and velocities ω . The input, $\mathbf{u}$, is defined as the base linear velocity in x, y , angular velocity in z , and the joint accelerations α . The system dynamics of the floating-base arm is defined in (5). We choose a first-order system to model the base dynamics since we observe that the locomotion policy's transient behavior in tracking the command velocity resembles a first-order system closely. In contrast, we choose second-order dynamics for the manipulator's joints. Note we can directly calculate the joint torques through the inverse dynamics of the arm, and these two representations are essentially identical.

$$\mathbf{f}_{\text{base}}(\mathbf{x}) = \begin{bmatrix} \dot{p}_x \\ \dot{p}_y \\ \dot{p}_z \\ \dot{r}_x \\ \dot{r}_y \\ \dot{r}_z \\ \dot{\theta}_{1,\dots,N} \\ \ddot{\theta}_{1,\dots,N} \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \\ -\kappa_1(p_z - h_{\text{des}}) \\ -\kappa_2 r_x \\ -\kappa_3 r_y \\ \omega_z \\ \omega_{1,\dots,N} \\ \alpha_{1,\dots,N} \end{bmatrix} \quad (5)$$

The system dynamics of the base, $\mathbf{f}_{\text{base}}$, is shown in (5). We assume perfect base velocity tracking in x, y , and yaw direction. The base height, roll, and pitch have passive first-order dynamics that decay to their desired values, where the coefficients κ are tuned from the trained student policy described in Section III-D2.

This simplified model captures the based behavior that is trained to horizontally align the base at a desired height.

The cost function for the end-effector tracking task consists of end-effector tracking cost, input cost, state deviation cost, which penalizes the arm joints for being away from their nominal values, and the joint velocity cost. In addition, we express joint position limits as inequality constraints. We solve the MPC problem defined with (3), (4), (5), and the above cost and constraints with Sequential Linear-Quadratic (SLQ) [19] using OCS2 [20].

Once the MPC solution is found, the planned state-input trajectory is used to calculate the sequence of joint torques and the wrench sequence exerted on the base using an inverse dynamic formulation. In our implementation, we use Pinocchio [21] for inverse dynamics.

During deployment on the robot, dynamics mismatch between the systems and the simplified model for the base motion (5) is inevitable. However, by running the MPC loop at a high rate (over 200 Hz), we ensure the state deviation from the planned trajectory is small.

C. Wrench Sequence Generator

During the RL policy training, the wrench exerted by the manipulator is simulated by the wrench sequence generator. Our wrench sequence generator consists of two models: Observed wrench and unobserved disturbances.

1) *Observed Wrench*: This component generalizes the smooth wrench from MPC plan as the arm moves. We generate this component by with polynomials to capture its smoothness. We define a randomized quadratic function for each wrench dimension over the two seconds after the current time.

$$w_{t=0}, w_{t=1.0}, w_{t=2.0} \sim \text{Uniform}(w_{\min}, w_{\max}) \quad (6)$$

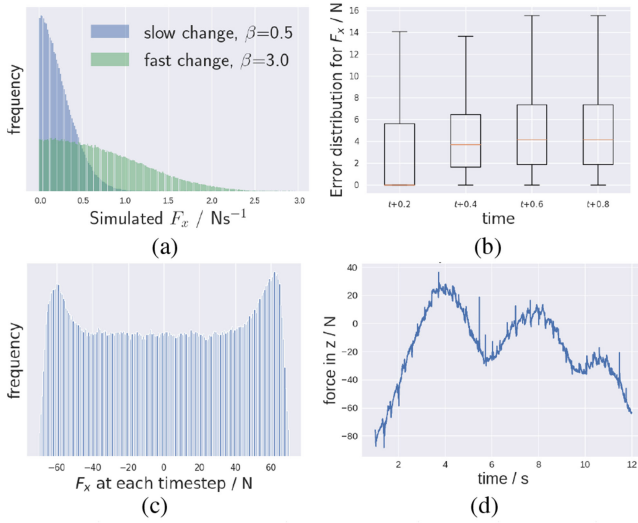


Fig. 3. Characteristics of the simulated wrench. (3a) The rate of change of force in x for different β . (3b) Box plot of the prediction error for force in x at time t in the future. (3c) The distribution of force in x that is applied on ANYmal. (3d) A sample of simulated force sequence in z .

$$w_{t+2.0+dt} \sim \text{Uniform}(w_{t+2.0} - \beta w_{\min}, w_{t+2.0} + \beta w_{\max}) \quad (7)$$

At the start of each episode, we sample wrench $w_{t=0}$, $w_{t=1.0}$, $w_{t=2.0}$ (6) and fit a quadratic function of t . Each sampled wrench w is a six-dimensional vector representing the external force and torque in 3-D. For each subsequent timestep with $dt = 0.02$ s, w_{t+dt} and $w_{t+1.0+dt}$ are calculated from the the polynomial fitted in the previous timestep, and a random $w_{t+2.0+dt}$ is sampled (7) and clipped between $w_{\min}$ and $w_{\max}$. β randomizes the rate of change of the polynomial term, which is also sampled per episode. The range of β values is tuned to produce smooth wrench that resemble those from the planned manipulator trajectories. w_t is applied to the robot in world frame. For each dimension, we calculate the wrench at $[0.0, 0.2, 0.4, 0.6, 0.8]$ after the current time t from the polynomial, and convert them to the robot base frame to feed them to the policy at each timestep during the training. The characteristics of the generated wrench is summarized in Fig. 3.

2) *Unobserved Disturbances*: In order to compute the induced arm wrench at the robot's base, we use inverse dynamics while ignoring the acceleration of the floating base. However, this assumption does not hold in practice due to the base motion, i.e., the cyclic pattern induced by the trotting gait. As a result, there is an unmodeled wrench component proportional to the inertia properties of the external loads: arm or other payloads. To make the policy robust against this component, we introduce a wrench disturbance induced by the base acceleration during the locomotion training. At the start of the episode, two random vectors v_{force} and v_{torque} are sampled uniformly in spheres centered at the origin. At each simulation timestep, we multiply v_{force} and v_{torque} with the robot's base linear and angular acceleration respectively (a_{base} and α_{base}) and apply the results on the robot as wrenches. Furthermore, we add Gaussian noise to this unobserved disturbance to simulate the remaining disturbances, such as cogging in the manipulator drives.

At each timestep of the simulation, the sum of the two simulated external wrench is applied to the geometric center of ANYmal's base. The RL agent receives the observed wrench prediction sequence in the form shown in (8), where w_t is the 6-d wrench queried at t seconds after the current time.

$$o_w = [w_0, w_{0.2}, \dots, w_{0.8}, \text{commands}, v_{\text{base}}, \omega_{\text{base}}]^T \quad (8)$$

The predicted wrench sequence is expressed in the body frame at time 0. The observed wrench and unobserved disturbance parameters are re-sampled at the start of each episode. A sample wrench sequence applied to the robot in the simulator is depicted in Fig. 3(d).

D. Base Policy Training

We follow the privileged training method proposed in [8]. We first train a teacher policy (Fig. 4(a)) with privileged observation. The privileged observation includes accurate states of the simulation environment, such as ground friction coefficient or ground reaction forces, which are not directly observable in the real world. Then we train a student policy (Fig. 4(b)) which imitates the teacher's behavior without the privileged information. The student policy learns to extract meaningful information from the sequence of the non-privileged observations.

The teacher policy's wrench observation consists of the polynomial observed wrench, the base velocity commands, and the base twist (i.e., linear and angular velocities). For the student policy, we additionally noisify the wrench observations. The perturbed wrench observation is concatenated with the proprioceptive observation and passed to the wrench Recurrent neural network (RNN) module, so that the policy may use the proprioceptive information to correct for the unobserved disturbances. In the case of quadrupedal robots, the teacher's policy converges to the trotting gait, and the student imitates this behavior. Readers can refer to [8] for the detailed format of the proprioceptive observation. The policies output the leg phases and a residual joint position. The phase here represents the progression of each foot in a cyclic stepping motion, and the residual joint position defines the offset joint angles of the joint-level PD controller. During the training, ANYmal is given a velocity command in x , y , and yaw direction for locomotion on rough terrain.

1) *Teacher*: The teacher agent is trained with Proximal Policy Optimization (PPO) [22] to maximize the expected cumulative trajectory rewards from the environment. The environment reward follows [10], with a higher weight on the base orientation, the body velocity in z , and the base angular velocity in pitch and roll.

2) *Student*: The student policy is distilled from the teacher policy by minimizing the loss function in (9), where $\mathcal{L}_{\text{action}}$ penalizes the difference between the student and teacher action, and $\mathcal{L}_{\text{embedding}}$ regresses the student wrench and exteroceptive embeddings towards those from the teacher.

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{action}} + \mathcal{L}_{\text{embedding}} + \mathcal{L}_{\text{decoder}} \quad (9)$$

$$\mathcal{L}_{\text{decoder}} = \mathcal{L}_{\text{privileged}} + \mathcal{L}_{\text{scan}} + \mathcal{L}_{w1} + \mathcal{L}_{w2} \quad (10)$$

In (10), $\mathcal{L}_{w1}$ is the scaled mean-squared error between the decoded wrench and the current external wrench applied to the robot base. $\mathcal{L}_{w2}$ regresses the decoded unobserved wrench

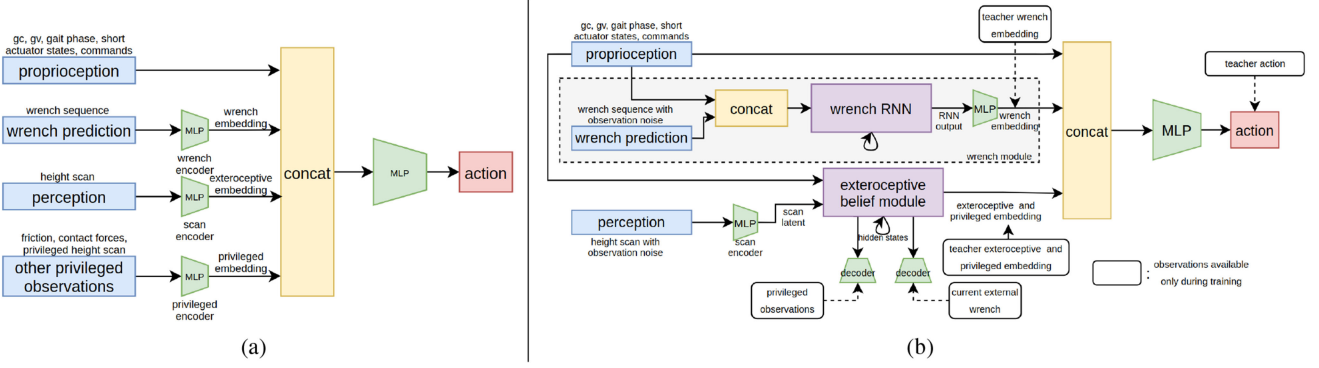


Fig. 4. (4a) Teacher policy structure. The teacher encodes the privileged observations with separate Multilayer Perceptron (MLP)s. gc: generalized coordinate, gv: generalized velocity. (4b) Student policy structure. The wrench module encodes the wrench prediction observation. The exteroceptive belief module is also trained to decode the external wrench from the proprioceptive history.

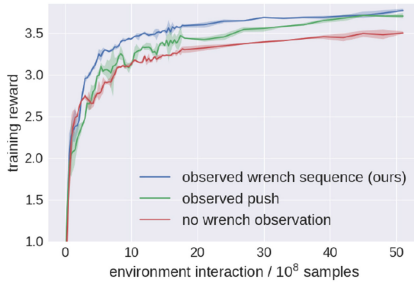


Fig. 5. Comparing training progress of the teacher policies.

characteristics towards the magnitude of v_{force} and v_{torque} in Section III-C. For manipulation tasks, the outputs from the decoder (e.g. load estimation) can be used for application purposes. We use two separate recurrent networks for the policy so that the decoder decodes the external wrench purely from proprioceptive and perceptive information without being interfered by the predicted wrench observations during the training.

IV. RESULTS

This section shows the experimental results of our approach. The video recordings of selected experiments are available in Movie 1.¹

A. Teacher Policy Performance

Fig. 5 compares the training progress of the teacher policy with and without the wrench prediction observations for five random seeds. It is clear from the curve that the external wrench prediction improves the teacher policy's learning progress and final performance. This shows that the teacher policy could indeed utilize the wrench prediction observations to counteract the disturbances. We additionally found that training with the smooth polynomial model as the predictable wrench results in a smoother learning progress than with intermittent pushes. We demonstrate with the other experiments in this section that the student policies can distill this ability.

¹Movie 1: <https://youtu.be/OZ9Adh0PYbw>

B. End-Effector Tracking

In this part, we deploy our proposed controller in simulation on ALMA to compare our method with the baselines for end-effector tracking. MPC receives a moving end-effector target at each timestep and produces the motion plan that includes the base velocity commands for the base RL policy, the arm joints trajectory, and predicted wrench induced at the base of the arm due to the arm motion. The performance of the base policy is measured with the mean absolute base tilt, the base angular velocity magnitude in pitch and roll, and the base linear velocity tracking error. We choose these evaluation metrics because they reflect the validity of the simplified base dynamics we described for MPC in Section III-B. Two experiment settings are used to measure the performance of the combined control system. In both settings, we used Dynaarm with a 3 kg end-effector. To improve consistency, we set ALMA to always trot in these experiments.

1) *Experiment 1*: On rough terrain, a random reference point is selected in a square of $3 \times 3 \text{ m}^2$, with height sampled from Uniform(0.6, 1.1) m. The reference point is re-sampled every five seconds. Each experiment runs for half an hour in simulation time. This experiment assesses the performance of the policies with relatively static motion plans since the reference target is not re-sampled frequently. The base moves close to the max speed command of 0.5 ms^{-1} when it is far away from the reference target.

2) *Experiment 2*: A reference point moves according to (11), where τ is the time since the start of the episode in seconds. The MPC commands the base to go to the origin with zero yaw.

$$p = [0.5, 0.5 \times \sin(\tau/0.6), 1.1]^T \quad (11)$$

The manipulator moves dynamically in this experiment as the end-effector oscillates in y . Compared to Experiment 1, the external wrench from the manipulator changes more quickly.

3) *Comparing Student Policy Structures*: We ablate the students trained with the following settings, Policies in this experiments are each trained with five seeds using the same teacher policy for the same number of environment steps, with the same rewards.

- *Ours*: The policy structure described in Section III-D2.

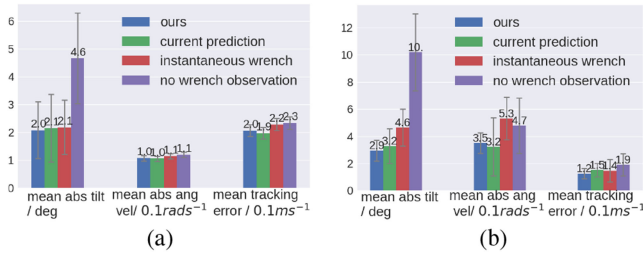


Fig. 6. Evaluation results of the tested student policy structures. (6a) Random end-effector targets. (6b) The end-effector target oscillates between the two sides of the robot.

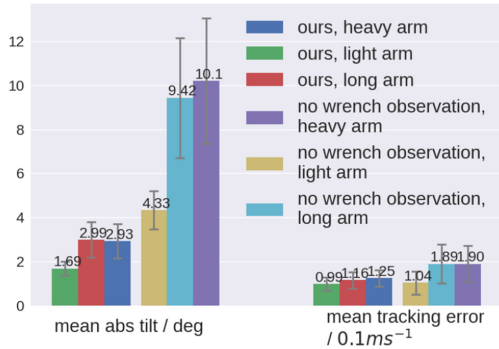


Fig. 7. Performance of the policy with different manipulator configurations.

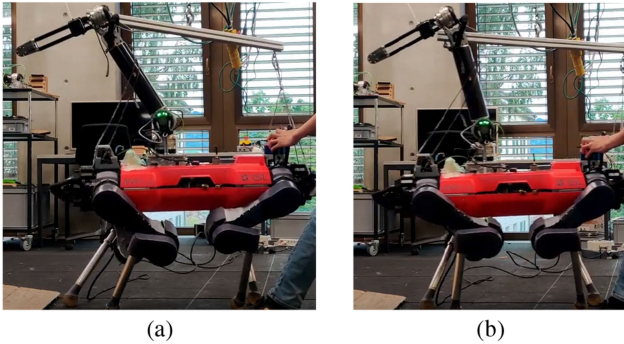


Fig. 8. Testing MPC's ability to track the desired end-effector position under base disturbances.

- *Current prediction*: Two layer of MLP is used instead of the RNN wrench encoder. The policy cannot utilize the history of past wrench predictions.
- *Instantaneous wrench*: Instead of using the wrench prediction sequence of 1 s, we only feed the wrench output calculated for the current timestep to the agent.
- *No wrench observation*: We remove the wrench module. The student policy only receives the proprioceptive and height scan observation.

Fig. 6(a) and 6(b) summarize the performance of different policy structures for the student policy. In general, it can be observed that the policies which utilize the external wrench predictions perform better. Close examination of these plots reveals a few interesting characteristics of our proposed controller.

4) *Explicit External Wrench Information is Beneficial*: Even in Experiment 1 that the manipulator has limited motion relative to the base, and the wrench is mainly from the weight of the manipulator, the external wrench improves the base tracking performance. Fig. 6(a) shows that all three policies with external wrench information have similar performance in base tilt and angular velocity, and they all outperform the policy with no external wrench observation. The mean tracking error assesses the base locomotion on rough terrain. The policy without wrench observation has a larger error because the base is more tilted, making it harder to walk.

5) *Wrench Prediction is Beneficial During Fast and Dynamic Motions*: In Experiment 2, the manipulator has larger motions with respect to the base, making it more challenging for the base to balance. The wrench prediction contains information about the wrench trends, and the base policy could utilize this information to counteract the wrench. In the experiment, the policy that only observes the current wrench from the manipulator tilts to the side more and takes longer to recover than the policies that have access to the wrench prediction sequence. This is shown in Fig. 6(b). In this experiment, the mean tracking error reflects the base's drift from the origin.

6) *Note*: In both experiments, we did not apply any constant offset or scaling to the wrench prediction, so the policy that uses the current wrench prediction is expected to perform similarly to the one with the RNN encoder. This is also consistent with our experiment results.

The advantage of the RNN encoder over the policy that only encodes the current prediction depends on the wrench noise magnitude during the training. A larger noise makes the RNN policy rely on the proprioceptive observation and deteriorates the overall policy performance. The improvement is visible on the hardware, but for the sake of consistency, we do not include this comparison.

C. Adaptation to Different Manipulator Configurations

We assess the adaptability of the combined control system to various manipulator configurations by testing Experiment 2 with two additional settings in simulation.

- *Light arm*: Dynaarm without any end-effector.
- *Long arm*: The forearm of Dynaarm is increased to 0.6 m, and the weight of forearm is adjusted accordingly. The end-effector is also removed in this setup.
- *Heavy arm*: Dynaarm with the 3 kg end-effector. This is the same setting as Experiment 2 above.

In this experiment, we compare our policy with the policy trained with no external wrench observation. When switching to manipulators that exert higher wrenches on the base, the policy that does not receive wrench observation tilts significantly more. In contrast, our policy is still able to keep the base close to horizontal (Fig. 7). This experiment shows that our policy can adapt to various manipulator configurations with a much smaller performance drop than the policy with no wrench observation.

D. End-Effector Tracking Under Base Disturbance

We disturb base motion manually while ANYmal is in stance to evaluate MPC's ability to track the desired end-effector position under disturbances. This hardware experiment showed a 57% reduction of the end-effector's position deviation compared

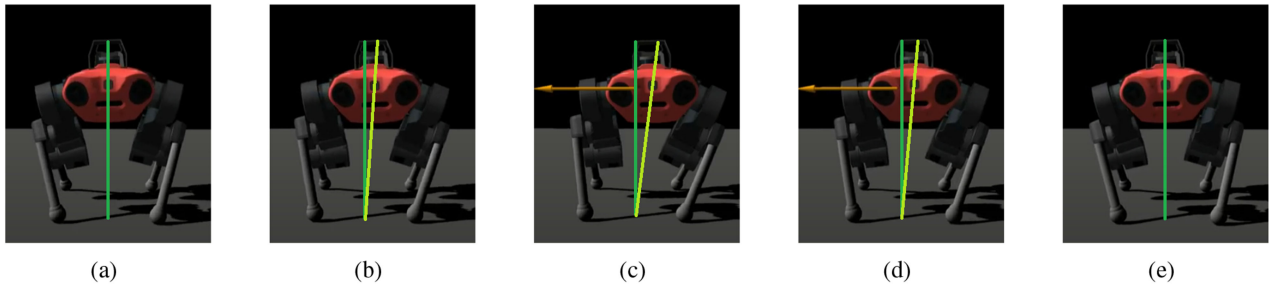


Fig. 9. Our proposed controller uses the wrench prediction to leaning against an incoming force of 100 N to maintain stability. (9a) Upright stance when no external force is anticipated. (9b) ANYmal leans right when it expects a force pulling from its left. (9c) ANYmal stands stably when the external force is applied. (9d)–(9e) ANYmal expects the pulling force to end, and returns to the upright posture.

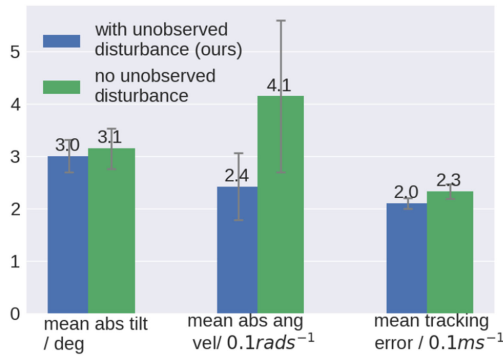


Fig. 10. Performance of the policies under dynamics mismatch.

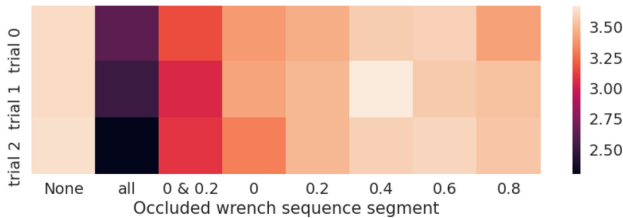


Fig. 11. Deterioration of the mean reward after occluding parts of the wrench observation.

to the base as we disturb the base in the x direction. This result is based on our default controller setting. The performance for disturbance rejection can be further improved by adjusting the cost formulation of MPC and the manipulator configuration.

E. Response to External Push: Anticipated Leaning

This experiment tests how our proposed controller can utilize the external wrench prediction to gain more robustness. To this end, we simulate a lateral force as a step function of time to the robot's base. The policy observes this external force as part of its predicted wrench input. We quantify the robot's stability as a maximum force that it can tolerate before stepping. As the policy receives a prediction of the incoming force, it starts leaning against it before the actual push happens in order to increase its stability. Fig. 9 shows a qualitative response. This behavior is compared to policies trained with the same teacher, but with different observation input: (1) Reactive Controller: it only

observes the current wrench estimation, (2) Naive Controller: it has no wrench observations.

In each test, we set ANYmal to the same joint configuration and the same gait phase at the start of the episode and apply a force in y -direction that lasts for 1 s. This experiment is deterministic for each policy, and student policies trained with five seeds are used for each setting to ensure reproducibility. We increase the force with the decimation of 10 N to record the maximum force that the policy can stabilize without stepping. Since our control policy can leverage the force prediction and adjust its posture before experiencing the force, the robot could stabilize under a larger force of 150 N without stepping to the side (170% of the reactive controller, and 208% of the naive controller). The policy that does not observe the external wrench could potentially counteract the force using the proprioceptive history. Still, the transient response is not as prompt as the policy that directly observes the current wrench.

F. Explicit Wrench Compensation

As described in Section III-D2, the RL agent is trained to decode the external wrench from proprioceptive information. This decoded output could be fed back to the RL actor and partially replace the external wrench prediction observation. In this simulated experiment, we compare ALMA's motion when using this explicit wrench compensation and discuss the advantage of using the wrench prediction instead of the decoded wrench.

Similarly to Experiment 2 in Section IV-B, we command the manipulator to move between the two sides of ALMA periodically to compare our proposed controller with the controller that replaces the wrench observation with the decoded wrench. The frequency that the arm moves cyclically is varied to evaluate the controllers. In this experiment, we use a 3 kg end-effector and command the arm shoulder rotation joint angle θ to move according to $\theta_{\text{SH_ROT}} = \frac{\pi}{2} \sin(\omega\tau)$, replacing the end-effector tracking cost with penalizing the deviation from commanded shoulder rotation for MPC. Table I indicates that using the decoded wrench to replace the MPC predictions results in a similar angular velocity tracking performance as our proposed controller when the arm moves slowly (when ω is below 2 rd s^{-1}). But when the manipulator undertakes fast and dynamic motions, the wrench prediction reduces the angular velocity error of the base and the chance of falling due to the disturbance.

TABLE I
COMPARISON BETWEEN USING THE WRENCH PREDICTION OBSERVATION AND
DECODED WRENCH

Angular frequency ω / rad s^{-1}		0	2	4	5
Base angular velocity error / (deg s^{-1})	ours	0.234	0.241	0.324	0.323
	decode	0.239	0.244	0.539	0.540
Average time before falling / s	ours	-	-	-	4.1
	decode	-	-	30	1.9

G. Dynamics Mismatch

This experiment assesses the importance of training under unobserved wrench disturbance for handling the dynamics mismatch during deployment. To this end, we train another policy that is not exposed to the base-acceleration induced term of the unobserved wrench during the training and compare its performance against our policy in a setting similar to Section IV-B1 Experiment 1 with an additional, unobserved 1 kg payload at the end-effector. Fig. 10 shows a comparison of our performance indices for these two policies. While we observe a reduction in tracking error for every metric, we note a significant improvement with respect to the mean angular base velocity error, which translates to less undesired body motion in practice.

H. Dependency on Wrench Observations

We occlude segments of the wrench observation to evaluate the policy's dependency on each part of the observation. The occlusion is implemented by replacing each dimension of the observation segment with a normal distribution with the running mean and variance from the normalizer for that dimension. We rollout the policy for one hour in simulation time in the training environment. Fig. 11 shows the deterioration of the mean step rewards. For our wrench observation format, occluding the imminent wrench prediction (0.0 s and 0.2 s in the future) leads to the most significant performance drop as expected. On average, the policy depends on the prediction at 0.8 s more than those at 0.4 s and 0.6 s. We hypothesize that this is because the predictions at 0.8 s are more informative than those at 0.4 s and 0.6 s as it evaluates the wrench trend further in the future.

V. CONCLUSION

This work presented a pipeline to combine learning-based locomotion policy with model-based manipulation for legged mobile manipulators. We showed that the wrench sequence predictions from MPC improve the base controller's performance on rejecting the external wrench. The combined controller adapts to a variety of manipulator configurations without re-training. Apart from manipulation, the policy is also suited for other applications such as load carrying and pulling. We demonstrated the controller on the robot hardware. To our knowledge, this is the first published result of legged mobile manipulation on rough terrain.

Currently, our base policy responds to external wrench passively and has no control over it. An opportunity for future work is to allow the base policy to have partial control by tracking its desired wrench with manipulators. We hypothesize that this

will enable more challenging maneuvers for the legged mobile manipulation systems.

REFERENCES

- [1] A. Gaweł *et al.*, "A fully-integrated sensing and control system for high-accuracy mobile robotic building construction," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 2300–2307.
- [2] M. Gifftthaler, F. Farshidian, T. Sandy, L. Stadelmann, and J. Buchli, "Efficient kinematic planning for mobile manipulators with non-holonomic constraints using optimal control," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, pp. 3411–3417.
- [3] Y. Abe, B. Stephens, M. P. Murphy, and A. A. Rizzi, "Dynamic whole-body robotic manipulation," *Proc. SPIE*, vol. 8741, 2013, Art. no. 87410V.
- [4] H. Dai, A. Valenzuela, and R. Tedrake, "Whole-body motion planning with centroidal dynamics and full kinematics," in *Proc. IEEE-RAS Int. Conf. Humanoid Robots*, 2014, pp. 295–302.
- [5] C. D. Bellicoso *et al.*, "Alma-articulated locomotion and manipulation for a torque-controllable robot," in *Proc. Int. Conf. Robot. Automat.*, 2019, pp. 8477–8483.
- [6] J.-P. Sleiman, F. Farshidian, M. V. Minniti, and M. Hutter, "A unified MPC framework for whole-body dynamic locomotion and manipulation," *IEEE Robot. Automat. Lett.*, vol. 6, no. 3, pp. 4688–4695, Jul. 2021.
- [7] J. Hwangbo *et al.*, "Learning agile and dynamic motor skills for legged robots," *Sci. Robot.*, vol. 4, no. 26, 2019, Art. no. eaau5872, doi: [10.1126/scirobotics.aau5872](https://doi.org/10.1126/scirobotics.aau5872).
- [8] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Sci. Robot.*, vol. 5, no. 47, 2020, Art. no. eabc5986, doi: [10.1126/scirobotics.abc5986](https://doi.org/10.1126/scirobotics.abc5986).
- [9] J. Tan *et al.*, "Sim-to-real: Learning agile locomotion for quadruped robots," in *Proc. Robot. Sci. Syst.*, Pittsburgh, Pennsylvania, Jun. 2018, doi: [10.15607/RSS.2018.XIV.010](https://doi.org/10.15607/RSS.2018.XIV.010).
- [10] T. Miki, J. Lee, J. Hwanbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Sci. Robot.*, vol. 7, no. 62, 2022, Art. no. eabk2822, doi: [10.1126/scirobotics.abk2822](https://doi.org/10.1126/scirobotics.abk2822).
- [11] X. B. Peng, P. Abbeel, S. Levine, and M. van de Panne, "Deepmimic: Example-guided deep reinforcement learning of physics-based character skills," *ACM Trans. Graph.*, vol. 37, no. 4, pp. 1–14, 2018.
- [12] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta, "Robust adversarial reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2817–2826.
- [13] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1126–1135.
- [14] M. Yin, G. Tucker, M. Zhou, S. Levine, and C. Finn, "Meta-learning without memorization," in *Proc. Int. Conf. Learn. Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=BkIEFpEYwS>
- [15] Z. Xie, X. Da, B. Babich, A. Garg, and M. van de Panne, "Glide: Generalizable quadrupedal locomotion in diverse environments with a centroidal model," 2021, *arXiv:2104.09771*.
- [16] V. Tsounis, M. Alge, J. Lee, F. Farshidian, and M. Hutter, "DeepGait: Planning and control of quadrupedal gaits using deep reinforcement learning," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 3699–3706, Apr. 2020.
- [17] M. Hutter *et al.*, "Anymal-a highly mobile and dynamic quadrupedal robot," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2016, pp. 38–44.
- [18] J. Hwangbo, J. Lee, and M. Hutter, "Per-contact iteration method for solving contact dynamics," *IEEE Robot. Automat. Lett.*, vol. 3, no. 2, pp. 895–902, Apr. 2018. [Online]. Available: www.raisim.com
- [19] F. Farshidian, M. Kamgarpour, D. Pardo, and J. Buchli, "Sequential linear quadratic optimal control for nonlinear switched systems," *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 1463–1469, 2017.
- [20] "OCS2: An open source library for optimal control of switched systems," [Online]. Available: <https://github.com/leggedrobotics/ocs2>
- [21] J. Carpentier *et al.*, "Pinocchio: Fast forward and inverse dynamics for poly-articulated systems," 2015–2021. [Online]. Available: <https://stack-of-tasks.github.io/pinocchio>
- [22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.